

Projetos em Ambientes Web e Cloud

Licenciatura em Tecnologias Digitais e
Inteligência Artificial

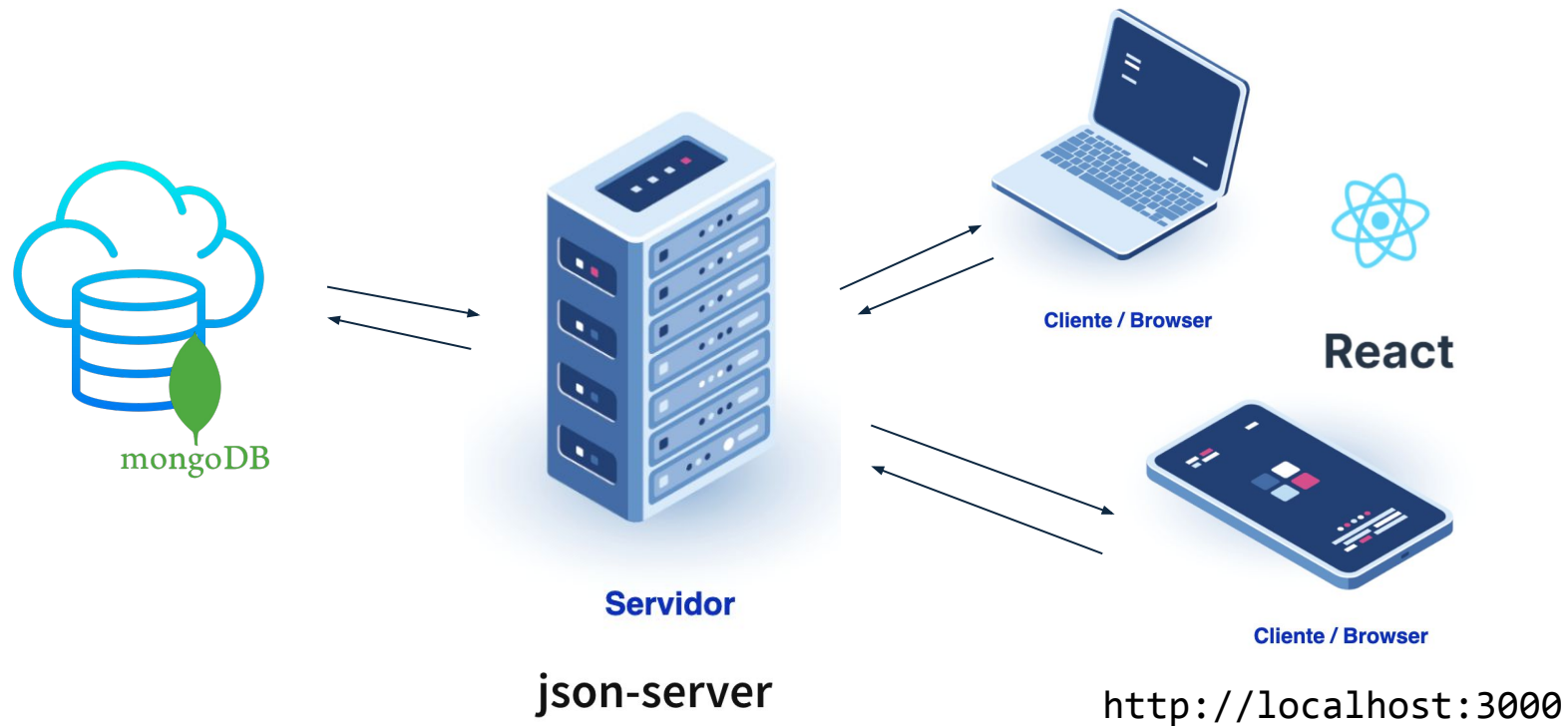
Aula 9

João Matos

joao_miguel_goncalves_matos@iscte-iul.pt

2º Semestre

Full-Stack



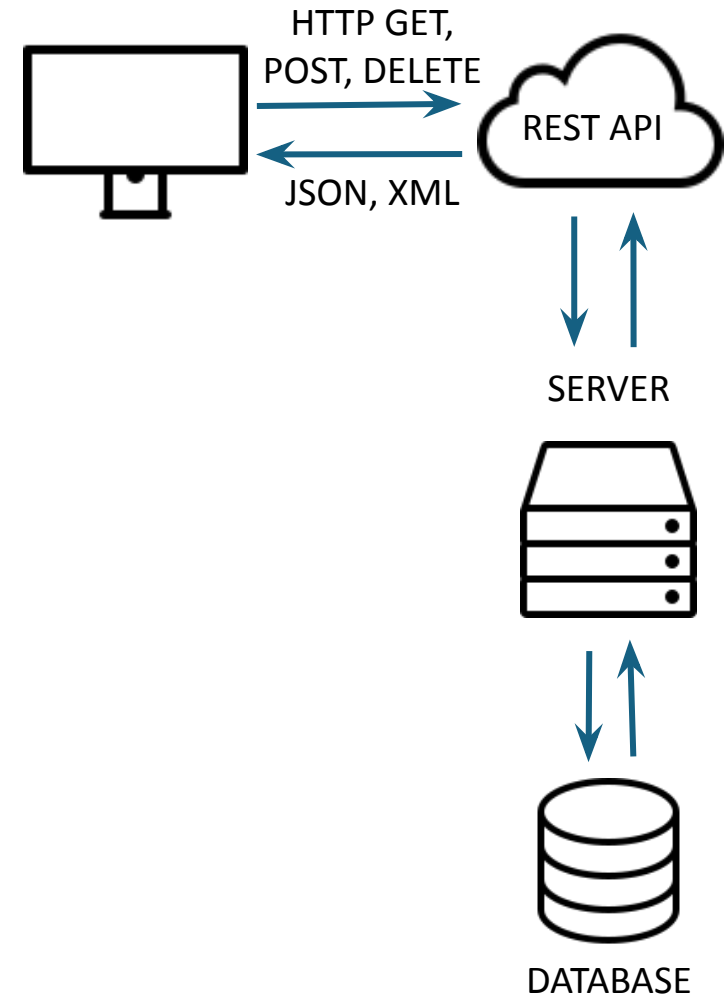
<http://localhost:3030/products>

<http://localhost:3030/favorites>

API – Application Programming Interface

Conjunto de regras e definições (comandos, funções e métodos) para aceder uma aplicação

- Interface que permite a comunicação entre diferentes sistemas de software;
- Facilita o pedido e envio de dados entre os sistemas;
- Mediador entre o cliente e o servidor;
- Representational State Transfer (REST)
 - Estilo arquitetural
 - Métodos HTTP (GET, POST, PUT, DELETE)
 - URLs para identificar recursos
 - Define operações CRUD (Create, Read, Update, Delete)



RESTful API

Estilo arquitetural que utiliza métodos HTTP padrão (GET, POST, PUT, DELETE) para realizar operações em recursos.

- Geralmente usa formato JSON (Javascript Object Oriented)
-> mensagens mais compactas e fáceis de processar
- Define URIs para identificar recursos
- Usa os métodos HTTP, o que permite definir operações sobre recursos; Operações CRUD (create, read, update, delete);

```
{
  "livros": [
    {
      "titulo": "Harry Potter e a Pedra Filosofal",
      "autor": "J.K. Rowling",
      "ano_publicacao": 1997,
      "genero": "Fantasia"
    },
    {
      "titulo": "O Senhor dos Anéis",
      "autor": "J.R.R. Tolkien",
      "ano_publicacao": 1954,
      "genero": "Fantasia épica"
    },
    {
      "titulo": "1984",
      "autor": "George Orwell",
      "ano_publicacao": 1949,
      "genero": "Ficção distópica"
    }
  ]
}
```

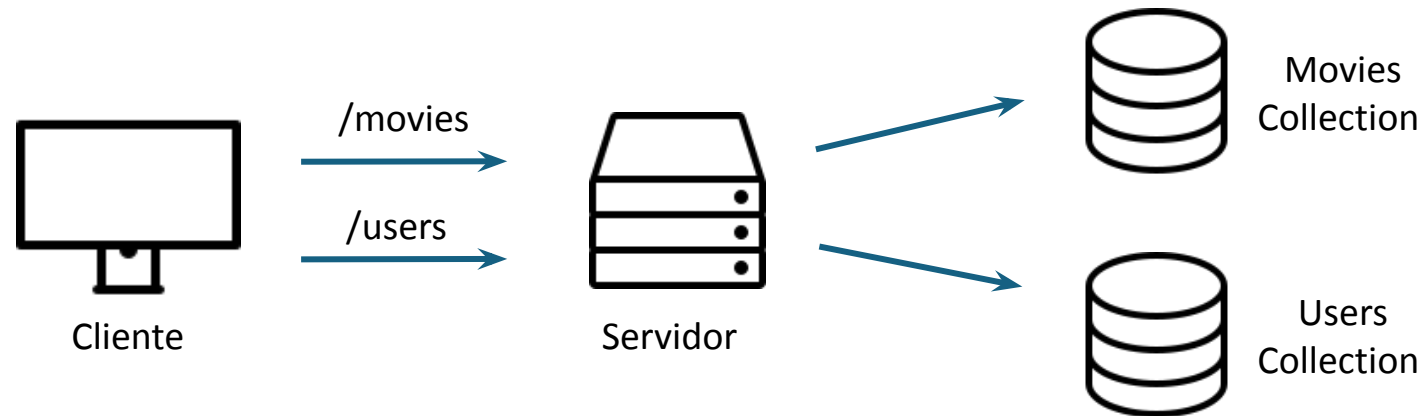
RESTful API

Organiza os diferentes *resources* em URIs únicos (Uniform Resource Identifiers)

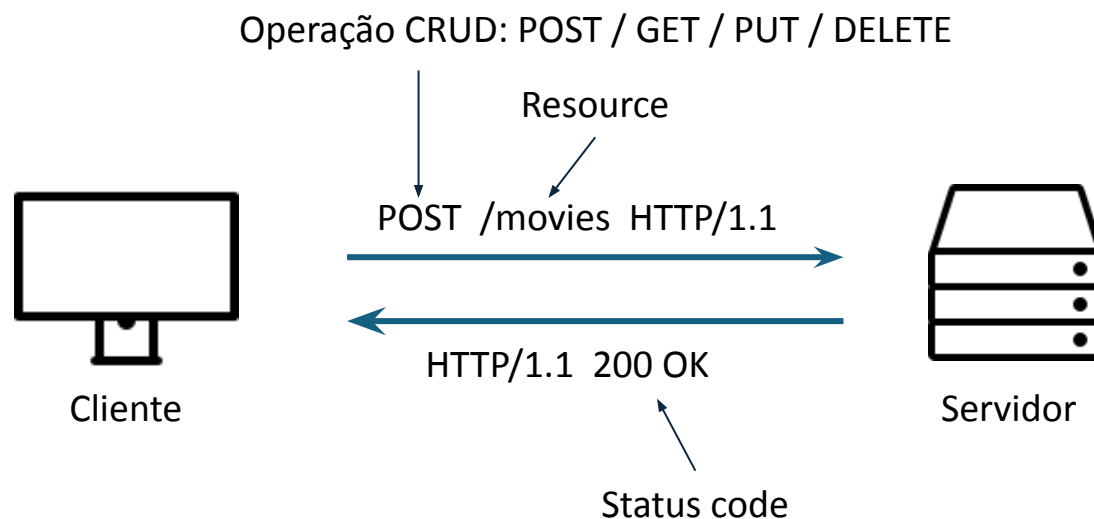
Cada URIs especifica o caminho de diferentes recursos no servidor

`http://host.com/api/v1/movies`

`http://host.com/api/v1/users`



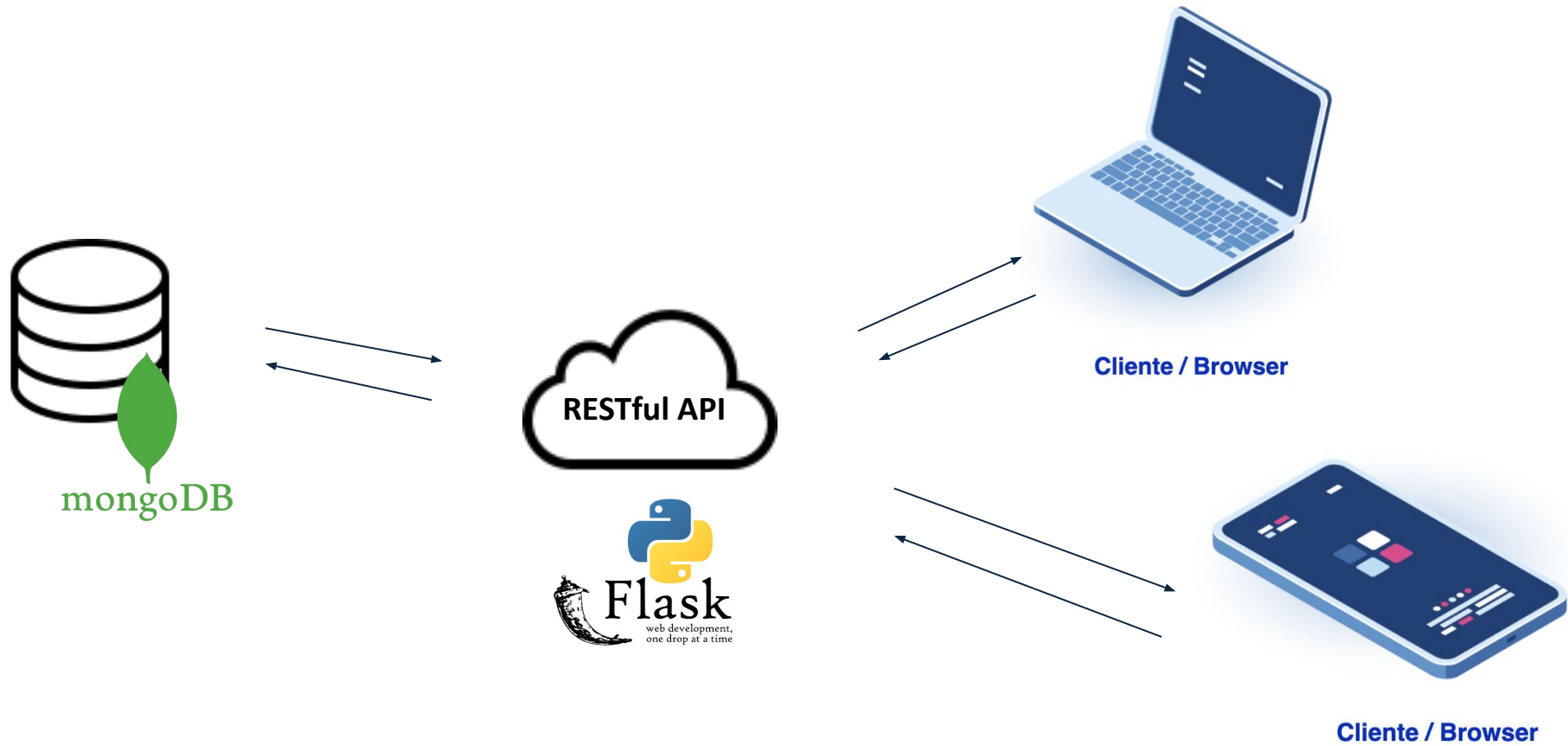
RESTful API



Request: Chamada ao endpoint através de HTTP - {operação e resource (endpoint)}

Response: Resposta do servidor a um recurso - {dados e “status code”}

RESTful API



Base de Dados Relacionais

- Armazena e organiza dados com relações definidas para um acesso rápido;
- Dados organizados em tabelas;
- Tabelas armazenam informações sobre cada entidade e representam categorias predefinidas através de **linhas** e **colunas**.;
- Estrutura de dados que permite o acesso eficiente e flexível.

Tabela Escritor				
ID_E	Nom	PaísResidência	DataNascimento	
2	Ana Magalhães	Portugal	06-10-1994	

Tabela Livro				
ID_L	Título	Preço	DataPublicação	Escritor
1	Verdadeira justiça	30	01-03-2015	2

Base de Dados Relacionais

- Linguagem SQL (Structured Query Language)
- DDL - Linguagem de descrição de dados: permite definir estrutura tabelas
 - CREATE / ALTER / DROP / TRUNCATE / RENAME
- DML - Data Manipulation Language: permite manipular dados:
 - SELECT / INSERT / UPDATE / DELETE

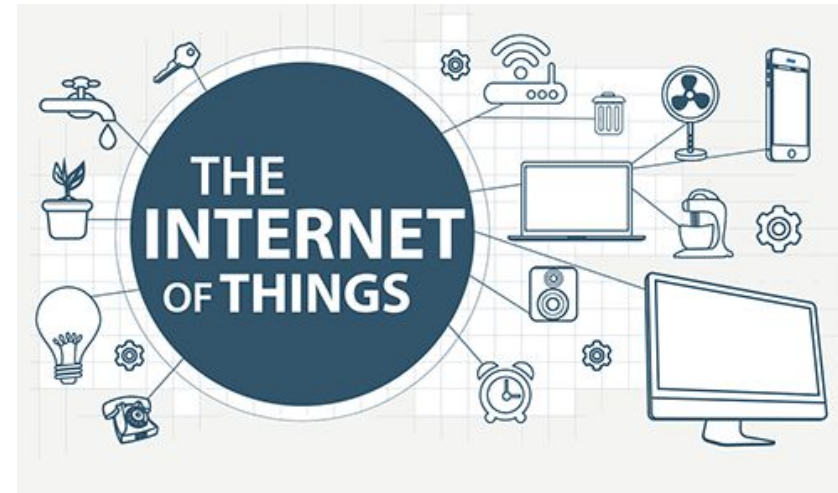
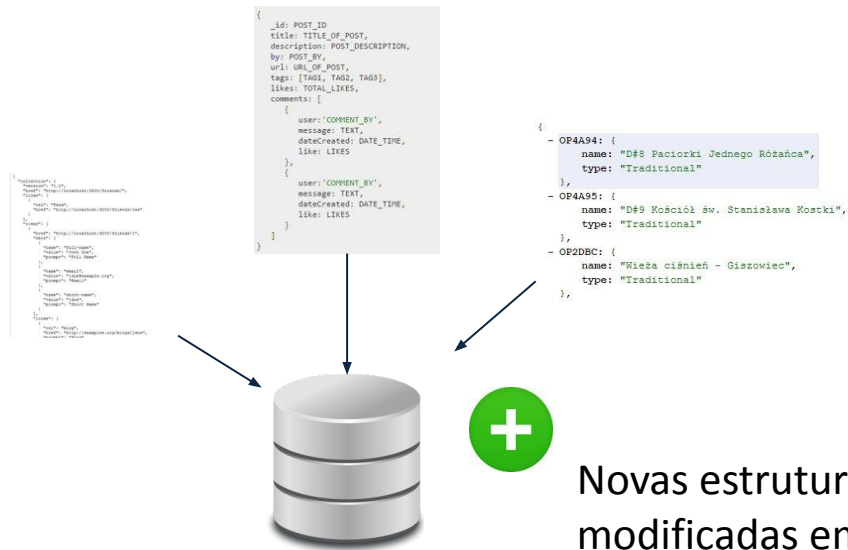
```
CREATE TABLE table_name
(
  column1 datatype [ NULL | NOT NULL ],
  column2 datatype [ NULL | NOT NULL ],
  ...
);
```

```
INSERT INTO table
(column1, column2, ... )
VALUES
(expression1, expression2, ... );
```

Aumento da quantidade de dados

O aumento dos “produtores” e quantidade de de dados (por exemplo, Internet of Things) conduz

- ao surgimento de novas soluções de armazenamento
- à necessidade de rapidamente (automaticamente) incorporar nas bases de dados estruturas de informação que não estavam previamente definidas



Novas estruturas de dados a serem criadas ou modificadas em tempo real

Flexibilidade

Tem de ser possível adicionar dados de novos produtores, sem que a aplicação seja interrompida ou reformulada.



Base de dados NoSQL

- Not Only SQL, ou modelos não relacionais;
- Representa um conjunto de sistemas de gestão de bases de dados que visa responder a um conjunto de necessidades mal resolvidas pelo modelo relacional, nomeadamente:
 - Flexibilidade
 - Escalonamento
- Não cumprem o mesmo conjunto de requisitos (ACID vs BASE)
- Os dados não são guardados em tabelas.
- Foco no desempenho e não na consistência dos dados. Exemplo: aplicação *social media*

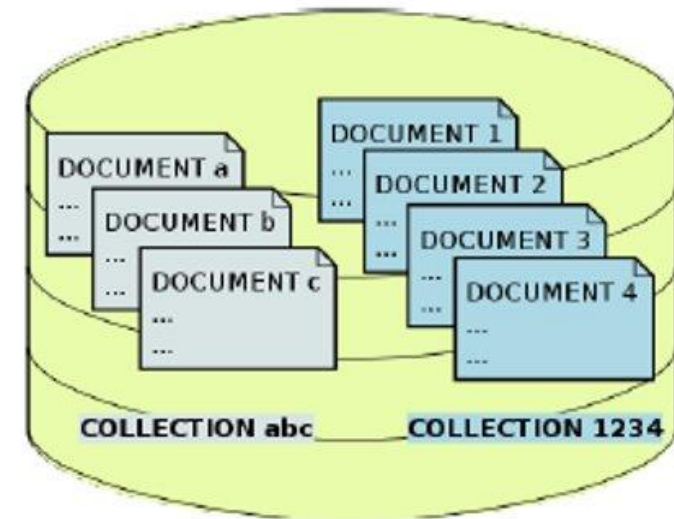
Modelo orientado a documentos



- Os documentos são estruturas de dados JSON (ou XML) agrupados em **coleções**
- Versão avançada de Key-Values que permite associar a uma chave estruturas mais complexas, tais como valores “encaixados” (*nested*) uns nos outros
- As **consultas são mais eficientes e mais ricas** que nas outras representações

Estrutura de um documento MongoDB

- Uma **coleção** de documentos equivale a uma **tabela** no modelo relacional
- Um **documento** equivale a uma **linha** no modelo relacional
- Um documento é um registo numa coleção de documentos
- Uma base de dados orientada a documentos é um conjunto de **coleções de documentos** armazenados
- O esquema não é necessariamente predefinido antes de se inserir dados



Formato JSON - Javascript Object Notation

- Formato compacto, do tipo **atributo-valor**
- Padrão **aberto** independente
- Transmissão de dados simples e rápida entre sistemas
- Especificado por Douglas Crockford em 2000
- Utiliza texto **legível** por humanos, **auto-descritivo**
- Derivado de **JavaScript**
- Muito usado em web services, usam transferência de estado representacional (REST), substituindo o uso do XML

```
{
  "cliente": {
    "id": 12345,
    "nome": "João Silva",
    "email": "joao.silva@example.com",
    "telefone": "(11) 99999-1234",
    "endereco": {
      "rua": "Rua das Flores",
      "numero": 150,
      "cidade": "Lisboa",
      "cep": "01001-000"
    }
  }
}
```

Sintaxe de um documento MongoDB

- **_id** é um identificador, gerado automaticamente ou não
- **att-1** é um atributo cujo valor é uma *string*
- **att-2** é um atributo cujo valor é um inteiro
- **att-3** é um atributo cujo valor é uma lista de valores
- **att-n** é um atributo cujo valor é um documento encaixado (*nested*)

```
{
  _id: <um identificador>,
  "att-1": "val-1",
  "att-2": val-2,
  "att-3": ["val-31", "val-32", ...]
  ...

  "att-n": {"val-n1": "val-n1",
            ....
           }
}
```


Document Operations

CRUD operations **C**reate, **R**ead,
Uppdate, and **D**elele documents

—————→ **IFUR** operations **I**nsert, **F**ind,
Uppdate, and **R**emove documents

Create Operations

```
db.collectionName.insertOne()  
db.collectionName.insertMany()
```

```
db.users.insertOne(  ← collection  
  {  
    name: "sue",      ← field: value  
    age: 26,          ← field: value  
    status: "pending" ← field: value  } document  
  }  
)
```

Reference: <https://www.mongodb.com/docs/manual/crud/>

- If the collection does not exists, MongoDB creates it during insert operations.
- No need to add '_id' field, MongoDB automatically assigns a unique id when we create a document

Document Operations

Read Operations - search documents inside a collection

```
db.collectionName.find()  
db.collectionName.findOne()
```

```
db.users.find(  
  { age: { $gt: 18 } },  
  { name: 1, address: 1 }  
) .limit(5)
```

← collection
← query criteria
← projection
← cursor modifier

- <query criteria>, opcional, um documento contendo os critérios/filtros de selecção dos documentos pertinentes
- <projection>, opcional, especifica os campos a retornar nos documentos que correspondem aos critérios de filtragem

Document Operations

Read Operations - Examples

```
db.collectionName.find({ <key>: <value> })
```

Examples:

```
db.collectionName.find( { name: "Joao" } )
```

```
db.collectionName.find( { age: 30 } )
```

Multiple criterias:

```
db.collectionName.find({ <key1>: <value1>, ... , <key_n>: <value_n>  })
```

Document Operations

Embedded/Nested documents:

```
db.users.find({  
  "access.group": "dev"  
})
```

```
{  
  _id: <ObjectId>,  
  username: "123xyz",  
  contact: {  
    phone: "123-456-7890",  
    email: "xyz@example.com"  
  },  
  access: {  
    level: 5,  
    group: "dev"  
  }  
}
```

Embedded sub-
document

Embedded sub-
document

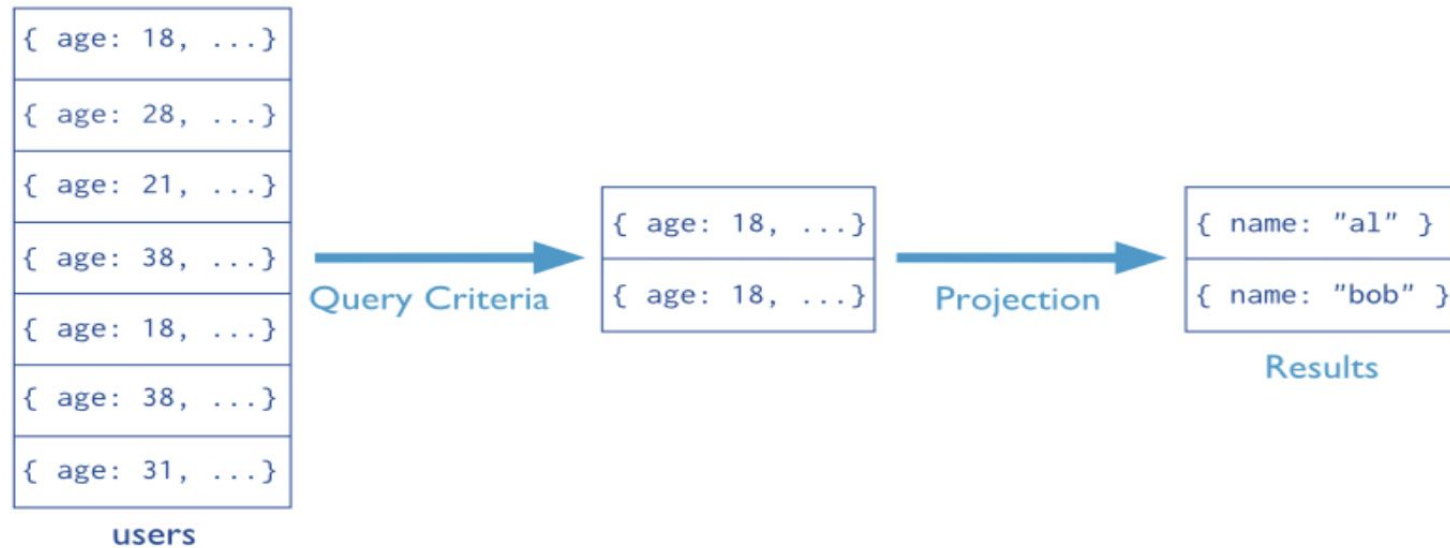
Document Operations

Query criteria and projection:

Projection:

```
{ FIELD_NAME: BOOLEAN }
```

Collection Query Criteria Projection
db.users.find({ age: 18 }, { name: 1, _id: 0 })



Document Operations

Logic operations: \$and, \$or, \$not, \$nor

\$and	Returns all documents that match the conditions of both clauses
\$or	Returns all documents that match the condition of either clause
\$not	Returns documents that do not match the query expression
\$nor	Returns all documents that fail to match both clauses

Document Operations

Comparison operations:

\$eq	Matches values that are equal to a specified value
\$gt	Matches values that are greater than a specific value
\$gte	Matches values that are greater than or equal to a specified value
\$in	Matches any of the values specified in an array
\$lt	Matches values that are less than a specified value
\$lte	Matches values that are less than or equal to a specified value
\$ne	Matches all values that are not equal to a specified value
\$nin	Matches none of the values specified in an array

Document Operations

Examples:

Documentos com preço igual a 5 e uma quantidade superior a 1:

```
db.collectionName.find({ $and: [ {preco: 5}, {quantidade: {$gt: 1}} ]})
```

Documentos com preço não superior a 5

```
db.collectionName({ preco: {$not: {$gt: 5}} })
```


Operações sobre documentos

Update Operations

```
db.collectionName.updateOne()  
db.collectionName.updateMany()
```

```
db.users.updateMany(  
  { age: { $lt: 18 } },  
  { $set: { status: "reject" } }  
)
```

← collection
← update filter
← update action

- Actualizar o campo “status” com o valor “reject” para todos os documentos onde “age” < 18

Operações sobre documentos

Delete Operations

```
db.collectionName.deleteOne()  
db.collectionName.deleteMany()
```

```
db.users.deleteMany(  
  { status: "reject" }  
)
```



collection
delete filter

Array Query Operations

\$all	Matches arrays that contains all elements specified in the query
\$elemMatch	Selects documents if element in the array field matches all the specified \$elemMatch conditions
\$size	Selects documents if the array field is a specified size

Array Query Operations

Operador `$all` retorna todos os documentos em que o *Array* atributo tem todos os valores especificados.

Syntax:

```
{ <field>: { $all: [ <value1> , <value2> ... ] } }
```

```
{ tags: [ "school", "book", "bag", "headphone", "appliance" ] }
```

```
> db.inventory.find( { tags: { $all: [ "appliance", "school", "book" ] } } )
```

Esta *query* retorna todos os documentos na coleção *inventory* em que o Array *tags* tem os valores 'appliance', 'school' e 'book'

Array Query Operations

Operador `$elemMatch` retorna todos os documentos em que o Array atributo tem pelo menos um elemento especificado na query

Syntax:

```
{ <field>: { $elemMatch: { <query1>, <query2>, ... } } }
```

```
{ "_id": 1, "results": [
  { "product": "abc", "score": 10 },
  { "product": "xyz", "score": 5 }
]},
{ "_id": 2, "results": [
  { "product": "abc", "score": 8 }
]},
{ "_id" : 3, "results": [
  { "product" : "abc", "score" : 7 },
  { "product" : "xyz", "score" : 8 }
] }
```

Pesquisar documentos onde o atributo 'results' tem pelo menos um element com 'product' igual a 'xyz' e 'score' maior ou igual a 8

```
db.survey.find(
  { results: { $elemMatch: { product: "xyz", score: { $gte: 8 } } } }
)
> { "_id" : 3, "results" : [ { "product" : "abc", "score" : 7 }, { "product" : "xyz",
"score" : 8 } ] }
```

Array Query Operations

The `$size` operator matches any array with the number of elements specified by the argument.

```
> db.collection.find( { field: { $size: 2 } } )
```

This query returns all documents in collection where field is an array with 2 elements.

Operações sobre documentos

Count method:

```
> db.client.find().count() , número de documentos encontrados na query  
> db.client.find({ age: 20 }).countDocuments()
```

Limit method:

```
> db.client.find().limit(<number>) , põe limite no número de resultados retornados  
> db.client.find({ age: 20 }).limit(1)
```

Sort method:

```
> db.client.find().sort({first_name: 1})  
ordena os documentos (1 for ascending order and -1 for descending order)
```

Group multiple methods

```
> db.client.find().sort({first_name: 1}).limit(5)
```

MongoDB: método skip()

Método de pesquisa que permite ignorar um número específico de documentos, antes de retornar os documentos da query.

Usado em sistemas com paginação.

```
db.collection.find().skip(10).limit(10)
```


MongoDB

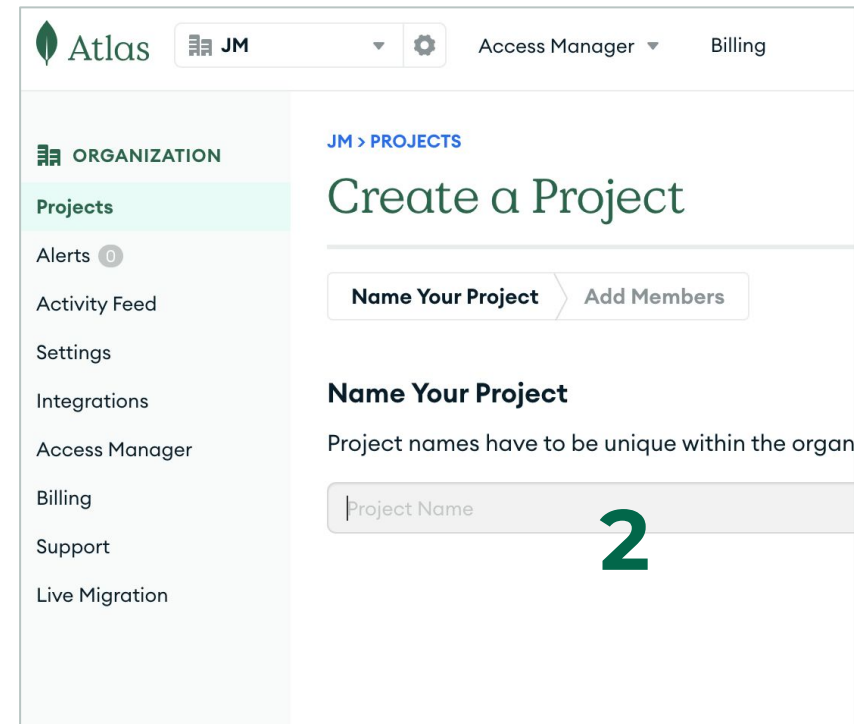
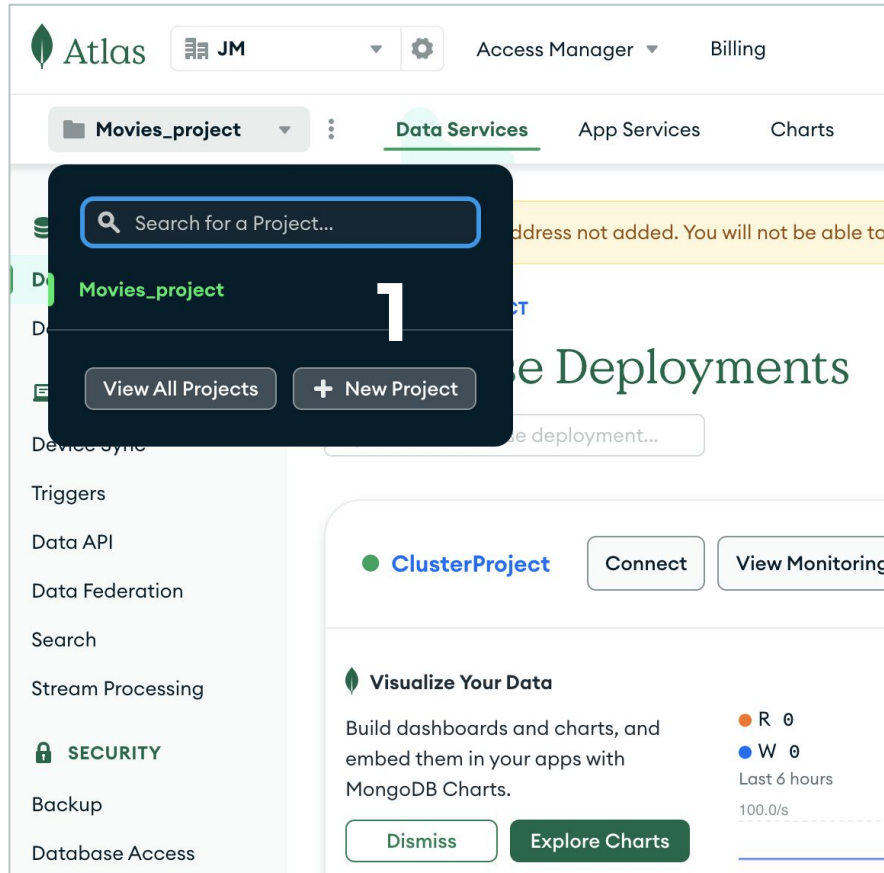
- Base de dados para MongoDB na *Cloud*
- Base de dados NoSQL
- Rápida instalação e *setup*

<https://www.mongodb.com/atlas>

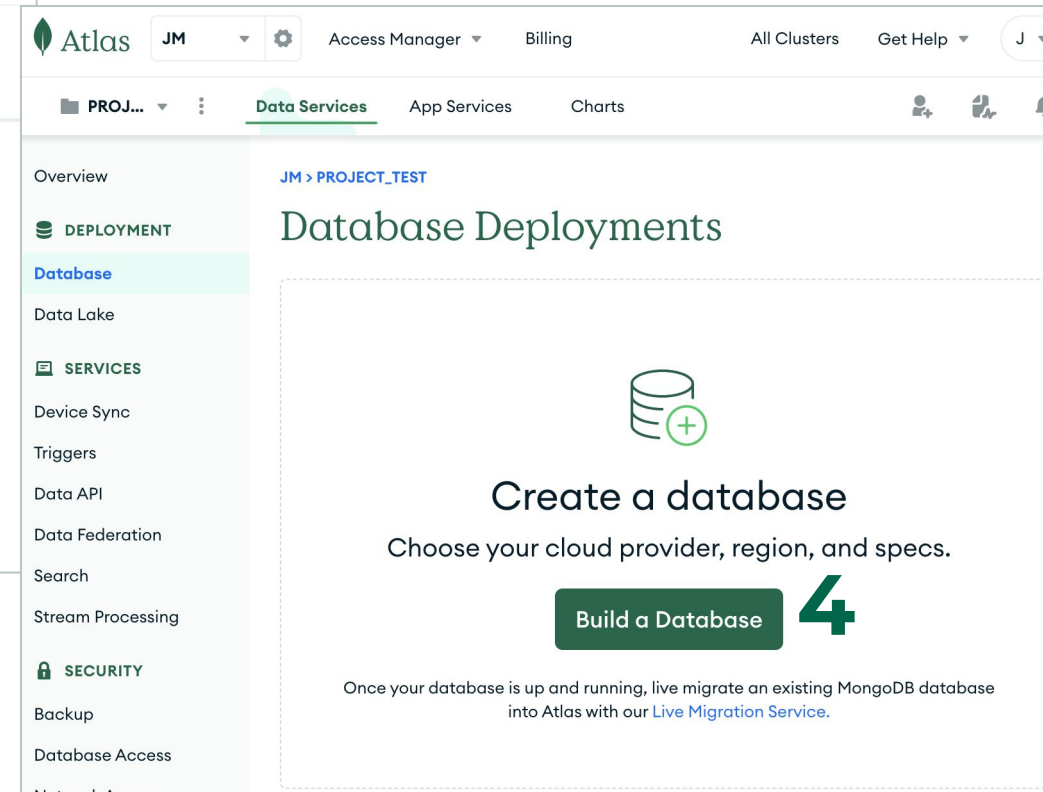
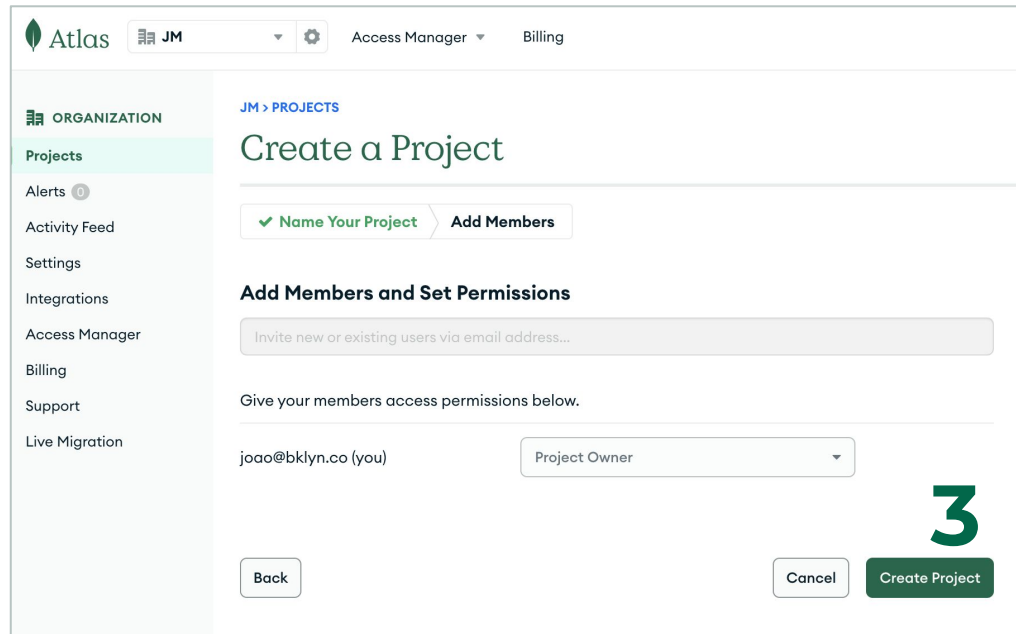
MongoDB Atlas. The multi-cloud developer data platform.

An integrated suite of cloud database and data services to accelerate and simplify how you build with data.

MongoDB Atlas: criar Base de Dados



MongoDB Atlas: criar Base de Dados



MongoDB Atlas: criar Base de Dados

M10 \$0.08/hour
For production applications with sophisticated workload requirements.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

SERVERLESS \$0.10/1M reads
For application development and testing, or workloads with variable traffic.

STORAGE	RAM	vCPU
Up to 1 TB	Auto-scale	Auto-scale

M0 FREE
For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

Provider: **aws** Google Cloud Azure

Region: ★ Recommended region
N. Virginia (us-east-1) ★

Name: ClusterProject

Tag (optional): Create your first tag to categorize and label your resources; more tags can be added later. [Learn more.](#)

FREE Create

Free forever! Your M0 cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime. [I'll deploy my database later](#)

[Access Advanced Configuration](#)

JM > PROJECT_TEST

Security Quickstart

To access data stored in Atlas, you'll need to create users and set up network security controls. [Learn more about security setup](#)

1 How would you like to authenticate your connection?

Your first user will have permission to read and write any data in your project.

Username and Password Certificate

1 We autogenerated a username and password for your first database user in this project using your MongoDB Cloud registration information. ✕

Create a database user using a username and password. Users will be given the *read and write to any database privilege* by default. You can update these permissions and/or create additional users later. Ensure these credentials are different to your MongoDB Cloud username and password.

Username

joao

Password

Enter password

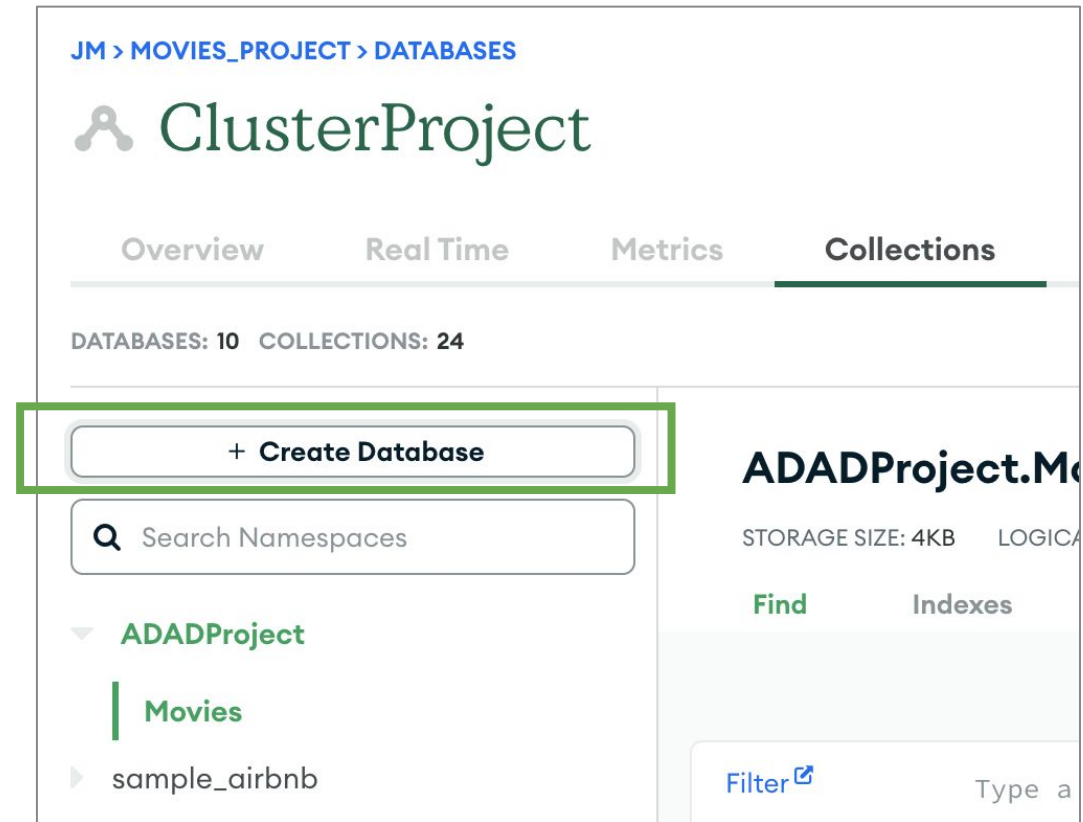
Autogenerate Secure Password Copy

Create User

MongoDB Atlas: criar Base de Dados

> Cluster > “Collections” Tab > “Create Database”

1. Criar base de dados “CloudProject”
2. Criar 2 coleções: “movies” e “users”

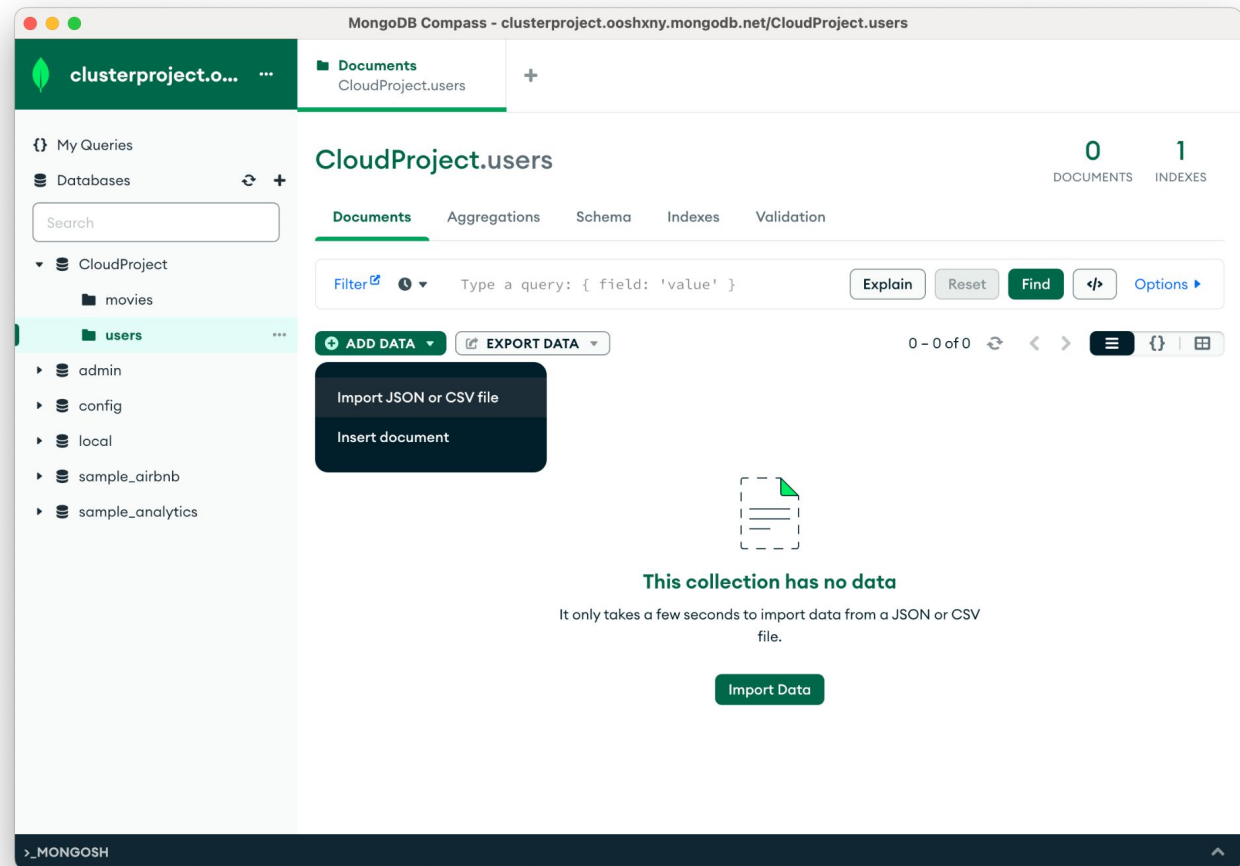


MongoDB Atlas: Compass

Download Compass:

<https://www.mongodb.com/try/download/compass>

> Import JSON files (movies.json e users.json)





Framework WSGI - Web Server Gateway Interface. Interface Python between applications and servers.

Implementação de *web applications* de forma eficaz, escaláveis e de fácil manutenção.

Permite receber dados do utilizador e enviar dados de volta dependendo do tipo de *request*

Python

Flask - *install e setup*

- Criar virtual environment:

Virtual environment, gestão de dependências do projeto.

Permite instalar bibliotecas específicas ao nosso projeto (pré-instalado com python 3.3)

```
> mkdir backend-app  
> cd backend-app  
> python3 -m venv venv
```

- Activar virtual environment:

Mac/Linux:

```
> source venv/bin/activate
```

Windows:

```
> venv\Scripts\activate
```


Flask - *install e setup*

Instalar Flask

> python3 -m pip install flask

- Criar ficheiro “app.py”
- Run:
 - > python app.py

```
from flask import Flask

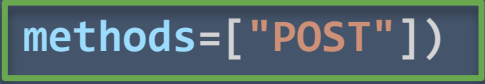
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, World!"

if __name__ == '__main__':
    app.run(debug=True)
```

Flask - *methods*

```
@app.route("/books", methods=["POST"])
def new_book():
    #adicionar novo livro
    return "POST REQUEST"
```



Define o método HTTP do *Route*:
(POST, GET, DELETE, PUT)

methods=["POST", "GET"]

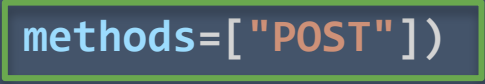
```
from flask import Flask, request

app = Flask(__name__)

@app.route('/books', methods=['GET', 'POST'])
def books():
    if request.method == 'POST':
        # (...)
        return "POST"
    else:
        # (...)
        return "GET"
```

Flask - *methods*

```
@app.route("/books", methods=["POST"])
def new_book():
    #adicionar novo livro
    return "POST REQUEST"
```



Define o método HTTP do *Route*:
(POST, GET, DELETE, PUT)

methods=["POST", "GET"]

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/books', methods=['GET', 'POST'])
def books():
    if request.method == 'POST':
        # (...)
        return "POST"
    else:
        # (...)
        return "GET"
```

Flask - *URL variables*

```
@app.route('/user/<username>')
def show_user_profile(username):
    # (...)
    return f'User {username}'

@app.route('/post/<int:post_id>')
def show_post(post_id):
    # (...)
    return f'Post {post_id}'
```

string	(default) any text without a slash
int	Accepts positive integers
float	accepts positive floating point values
path	like string but also accepts slashes

Flask - URL parameters

http://host.com/api/v1/movies?type=comedy&year=2000

```
@app.route('/movies', methods=['POST'])
def get_query_example():

    type = request.args.get('type')
    year = request.args.get('year')

    return (...)
```

Flask + MongoDB

PyMongo: Interface MongoDB.

- Permite estabelecer conexões com servidores MongoDB
- Manipulação de dados MongoDB: insert / find / update / delete

```
from flask import Flask
from pymongo import MongoClient

app = Flask(__name__)

client = MongoClient(<MONGO_URL_STRING>,
                    tls=True,
                    tlsAllowInvalidCertificates=True)
db = client[<DATABASE_NAME>]
```

<https://pymongo.readthedocs.io/en/stable/index.html>

Instalar PyMongo:

```
> python3 -m pip install pymongo
```

Flask + MongoDB

```
from flask import Flask
from pymongo import MongoClient

app = Flask(__name__)

client = (...)
db = client[<DATABASE_NAME>]

@app.route("/users", methods=["GET"])
def get_users():
    users = db.users.find().limit(10)
    cursor = list(users)
    return jsonify(cursor)
```

```
> collection.insert_one({"key": "value"})
> collection.insert_many([
    {"key1": "value1"},
    {"key2": "value2"}
])

> cursor = collection.find({"key": {"$gt": 5}})

> cursor = collection.find().sort("key", pymongo.ASCENDING)
# ou DESCENDING

> collection.update_one(
    {"key": "value"},
    {"$set": {"new_key": "new_value"}}
)
> collection.update_many(
    {"key": "value"},
    {"$set": {"new_key": "new_value"}}
)

count = collection.count_documents({"key": "value"})
```